

Week08_Ch06(합성곱 신경망2 - 6.1.5~)

6.1. 이미지 분류를 위한 신경망

- 입력 데이터로 이미지를 사용한 분류: 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것

이미지 분류에서 주로 사용되는 합성곱 신경망의 유형

6.1.5. ResNet

6.1.5. ResNet

✓ ResNet 개념

- 마이크로소프트에서 개발한 알고리즘
- 핵심: 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼 개념을 고안한 것

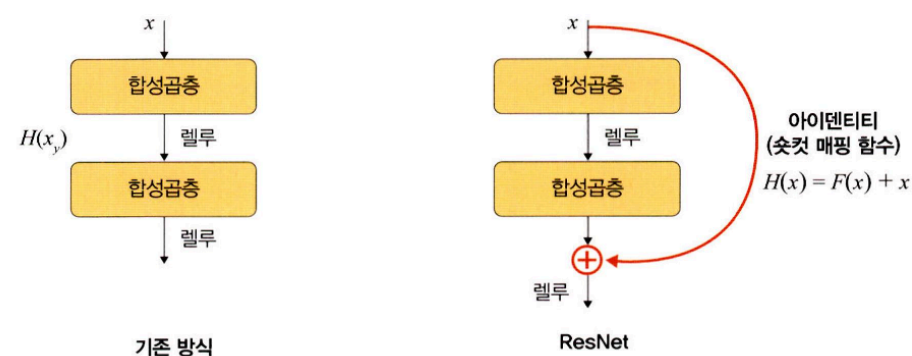
1) 신경망의 깊이와 성능

- 신경망은 깊이가 깊어질수록 성능이 좋아지다가 일정한 단계에 다다르면 오히려 성능이 나빠짐
- 네트워크의 깊이가 깊다고 해서 무조건 성능이 좋아지지는 않음

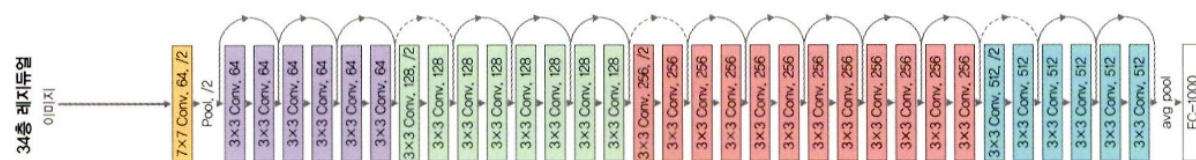
⇒ ResNet은 이러한 문제를 해결하기 위해 레지듀얼 블록을 도입

⇒ 레지듀얼 블록은 기울기가 잘 전파될 수 있도록 일종의 지름길을 만들어 줌

⇒ 기울기 소멸 문제를 방지



2) 블록



- 계층의 묶음
- 합성곱층을 하나의 블록으로 묶은 것

⇒ 레지듀얼 블록

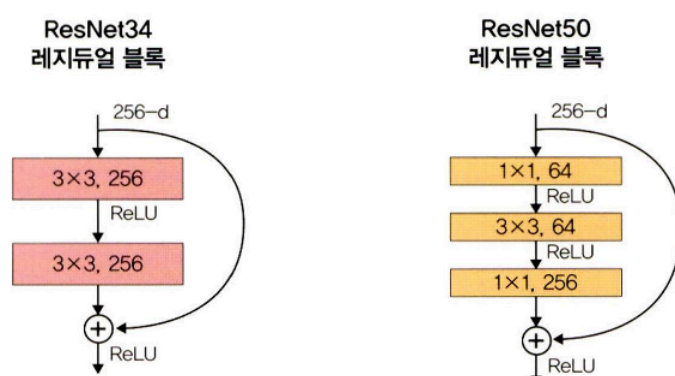
⇒ 레지듀얼 블록들을 쌓은 것이 ResNet

문제점

- 계층들을 계속해서 쌓아 늘리면(깊어질수록) 파라미터가 증가

⇒ 이러한 문제를 해결하기 위해 병목블록(bottleneck block)을 줌

3) 병목 블록



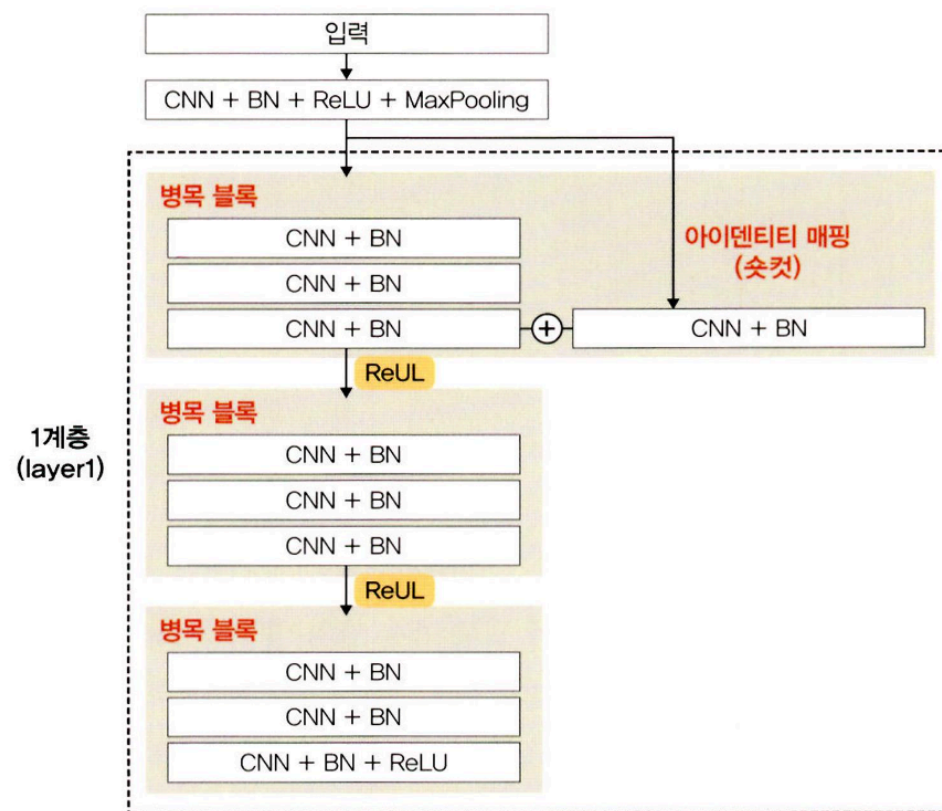
- ResNet34: 기본 블록 사용, 파라미터 39.3216M

- ResNet50: 레지듀얼 블록 사용, 파라미터 6.9632M

⇒ 깊이가 깊어졌음에도 파라미터 수는 감소

- 1×1 합성곱층의 채널 수를 조절하면서 차원을 줄였다 늘리는 것이 가능
 - ⇒ 파라미터 수를 줄일 수 있음
 - ⇒ 병목 블록

4) identity mapping/shortcut/skip connection



- 아이덴티티 매핑(+): 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되도록 함

코드

```
def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

    if self.downsample is not None:
        i = self.downsample(i) # 다운샘플 적용

    x += i # 아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

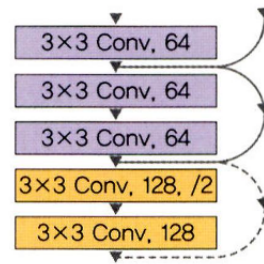
- 입력 x 를 i 라는 변수에 저장
- 입력 x 는 합성곱층을 통과하다가 마지막 x 에 i 를 더함
- x 가 (28, 28, 64)라고 가정
- x 를 i 변수에 저장했기 때문에 (28, 28, 64)가 됨
- 합성곱층을 통과하면서 같은 형태를 더하기 때문에 최종 형태는 그대로 (28, 28, 62)가 됨

5) 다운샘플(downsample)

- 특성맵 크기를 줄이기 위한 것

- 풀링과 같은 역할

▼ 그림 6-29 ResNet 네트워크의 일부

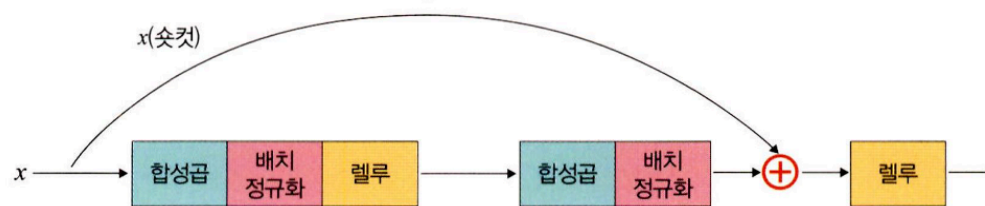


- 보라색 영역의 첫번째 블록에서 특성 맵의 형상이 (28, 28, 64)였다면 세 번째 블록의 마지막 합성곱층을 통과하고 아이덴티티 매핑까지 완료된 특성 맵의 형상도 (28, 28, 64)임
- 노란색 영역의 시작 지점에서는 채널 수가 128로 늘어났고, /2라는 것으로 보아 첫 번째 블록에서 합성곱층의 스트라이드가 2로 늘어나 (14, 14, 128)로 바뀜
- 보라색과 노란색의 형태가 다른데, 이들 간의 형태를 맞추지 않으면 아이덴티티 매핑을 할 수 없게 됨
⇒ 아이덴티티에 대해 다운샘플이 필요

6) 프로젝션 숏컷/합성곱 블록

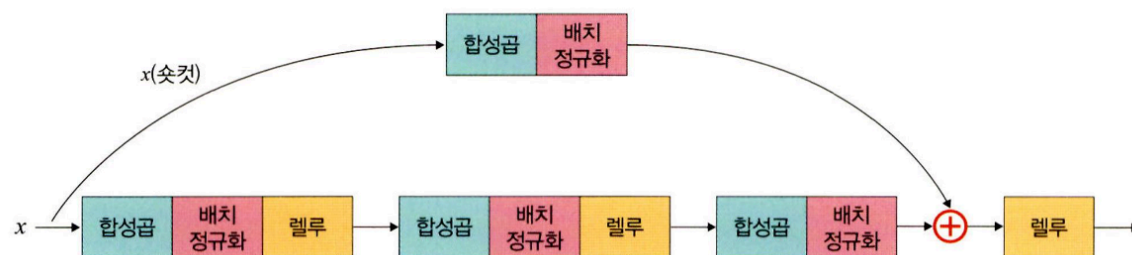
- 입력과 출력의 형태를 같도록 맞추어 주기 위해서는 스트라이드 2를 가진 1x1 합성곱 계층을 하나 연결하면 됨
- **아이덴티티 블록:** 입력과 차원의 출력이 같은 것

▼ 그림 6-31 아이덴티티 블록



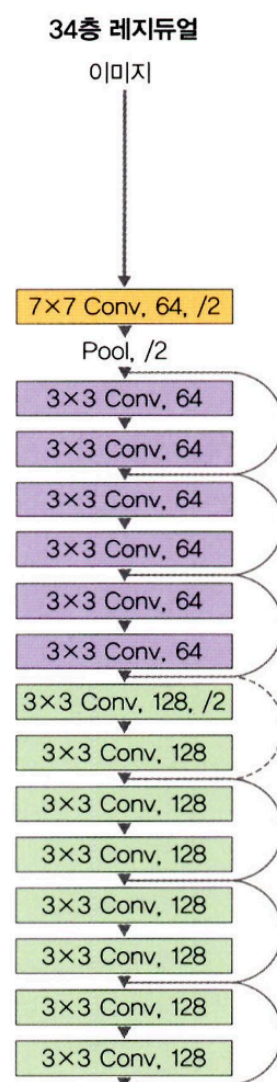
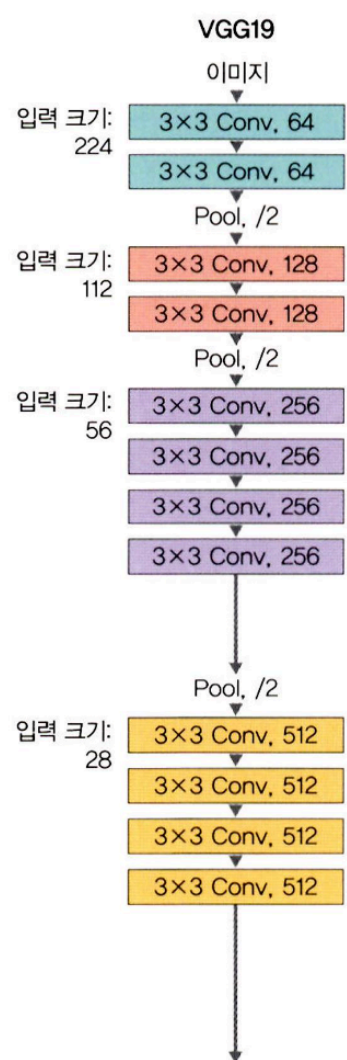
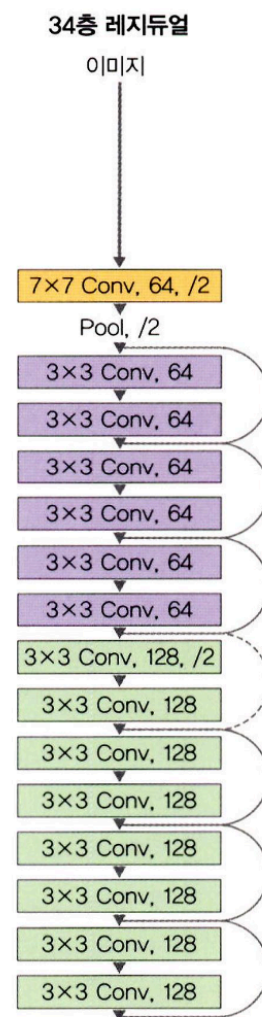
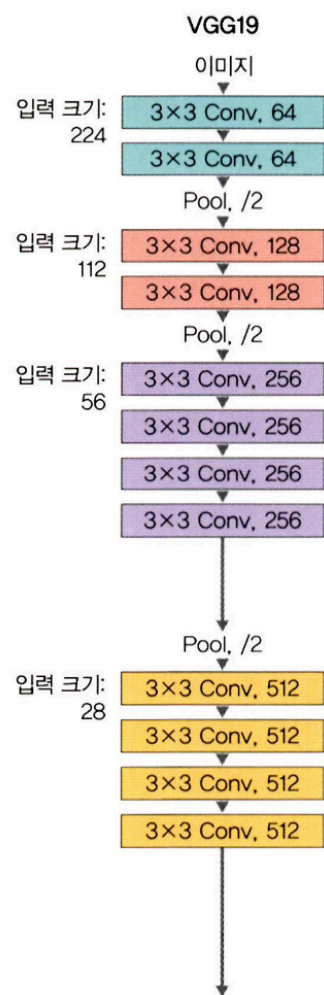
- **프로젝션 숏컷/합성곱 블록:** 입력 및 출력의 차원이 동일하지 않고 입력의 차원을 출력에 맞추어 변경해야 하는 것

▼ 그림 6-30 합성곱 블록



✓ ResNet 구조

- 기본적으로 VGG19 구조를 뼈대로 하며, 거기에 합성곱층들을 추가해서 깊게 만든 후 숏컷들을 추가



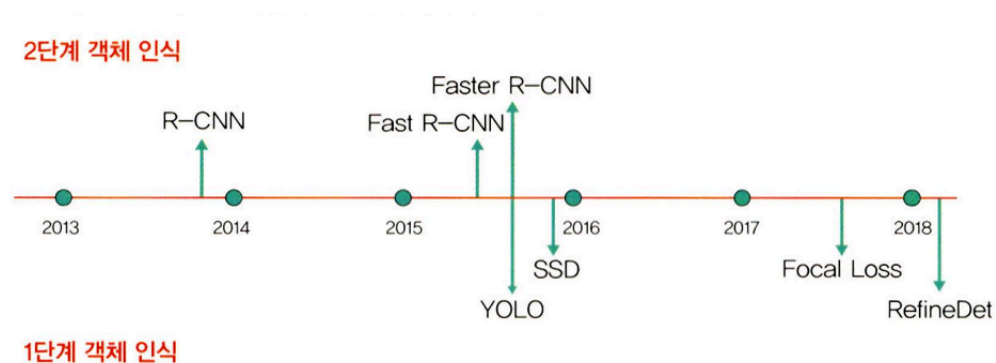
✓ 파이토치로 ResNet 구현

6.2. 객체 인식을 위한 신경망

6.2.0. 객체인식

- 이미지, 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술
- 이미지, 영상 내에 있는 여러 객체에 대해 각 객체가 무엇인지 분류하는 문제 + 그 객체 위치가 어디인지 박스로 나타내는 위치 검출 문제

딥러닝을 이용한 객체 인식 알고리즘



- 1단계 객체 인식
 - 두 문제(분류, 위치 검출)를 동시에 행하는 방법
 - 비교적 빠르지만 정확도가 낮음
 - YOLO 계열, SSD 계열 등이 포함
- 2단계 객체 인식
 - 두 문제를 순차적으로 행하는 방법
 - 비교적 느리지만 정확도가 높음
 - R-CNN 계열(CNN을 처음으로 적용)이 대표적

6.2.1. R-CNN

과거 객체 인식 알고리즘

- 슬라이딩 윈도우 방식으로 이미지의 모든 영역을 탐색하면서 객체를 검출

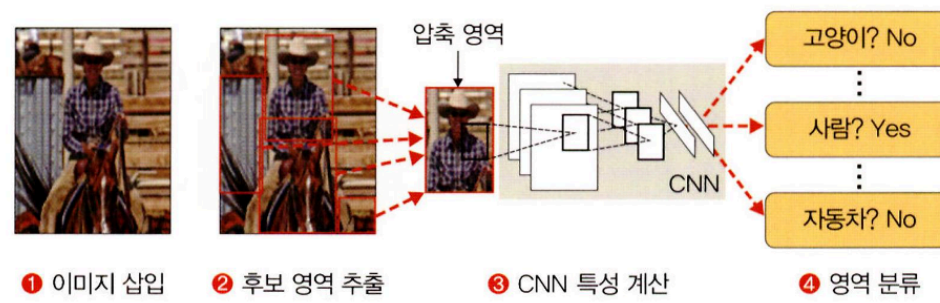
⇒ 알고리즘의 비효율성 때문에 많이 사용하지 않았음

⇒ 현재는 선택적 탐색(selective search) 알고리즘을 적용한 후보 영역(region proposal)을 많이 사용

R-CNN의 개념

- 이미지 분류를 수행하는 CNN + 이미지에서 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘

수행 과정



1. 이미지를 입력으로 받음
2. 2000개의 바운딩 박스를 선택 → 선택적 탐색 알고리즘으로 추출 → 잘라냄 → CNN 모델에 넣기 위해 같은 크기(227x227 픽셀)로 통일
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용
4. 각각 분류를 진행하여 결과를 도출

선택적 탐색

• 개념

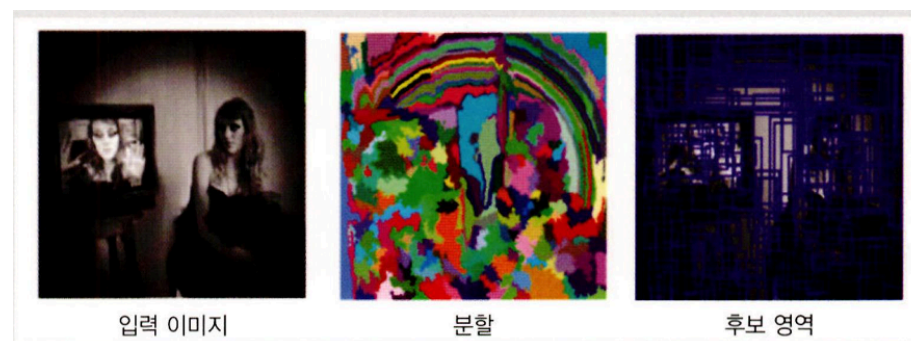
- 객체 인식, 검출을 위한 가능한 후보 영역(객체가 있을 만한 위치, 영역)을 알아내는 방법
- 분할 방식을 이용하여 시드(seed)를 선정 → 시드에 대한 완전 탐색을 적용

• 과정

1. 초기 영역 생성(sub-segmentation)

: 각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기 영역을 생성

⇒ 입력된 이미지를 영역 다수 개로 분할하는 과정



2. 작은 영역의 통합

: 1단계에서 영역 여러 개로 나눈 것들을 비슷한 영역으로 통합. 이때 탐욕 알고리즘을 사용하여 비슷한 영역이 하나로 통합될 때까지 반복



3. 후보 영역 생성

: 2단계에서 통합된 이미지들을 기반으로 다음 그림과 같이 후보 영역 (바운딩 박스)을 추출



- **완전 탐색(exhaustive search):** 후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황을 고려하여 후보 영역을 찾는 기법
- **분할(segmentation):** 영상 데이터의 특성(색상, 모양, 무늬 등)에 따라 분할하여 후보 영역을 선정하는 기법
- **후보 영역(바운딩 박스):** 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
- **시드(seed):** 영상에서는 특정 기준점의 픽셀에서 점점 의미가 같은 영상 범위까지 픽셀을 확장해 나가면서 분할하는데, 이때 특정 기준점이 되는 픽셀

단점

- 앞서 언급한 세 단계의 복잡한 학습 과정
- 긴 학습 시간과 대용량 저장 공간
- 객체 검출 속도 문제

⇒ 이러한 문제를 해결하기 위해 Fast R-CNN이 생김

6.2.2. 공간 피라미드 풀링

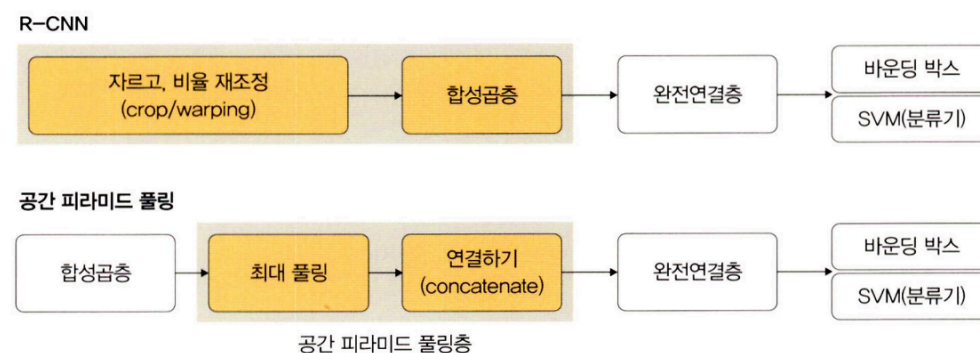
기존 CNN 구조

- 완전연결층을 위해 입력 이미지를 고정해야 함
- 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나 비율을 조정해야 함

⇒ 물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제점이 있음

⇒ 이러한 문제를 해결하고자 공간 피라미드 풀링(spatial pyramid pooling)을 도입

공간 피라미드 풀링의 개념



- 입력 이미지의 크기와 관계없이 합성곱층을 통과시키고, 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법

장점

- 입력 이미지의 크기를 조절하지 않고 합성곱층을 통과시킴
⇒ 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있음
- 이미지 분류, 객체 인식 같은 여러 작업에 적용할 수 있음

6.2.3. Fast R-CNN

R-CNN의 한계

- 바운딩 박스마다 CNN을 돌리고, 분류를 위한 긴 학습 시간이 소요됨

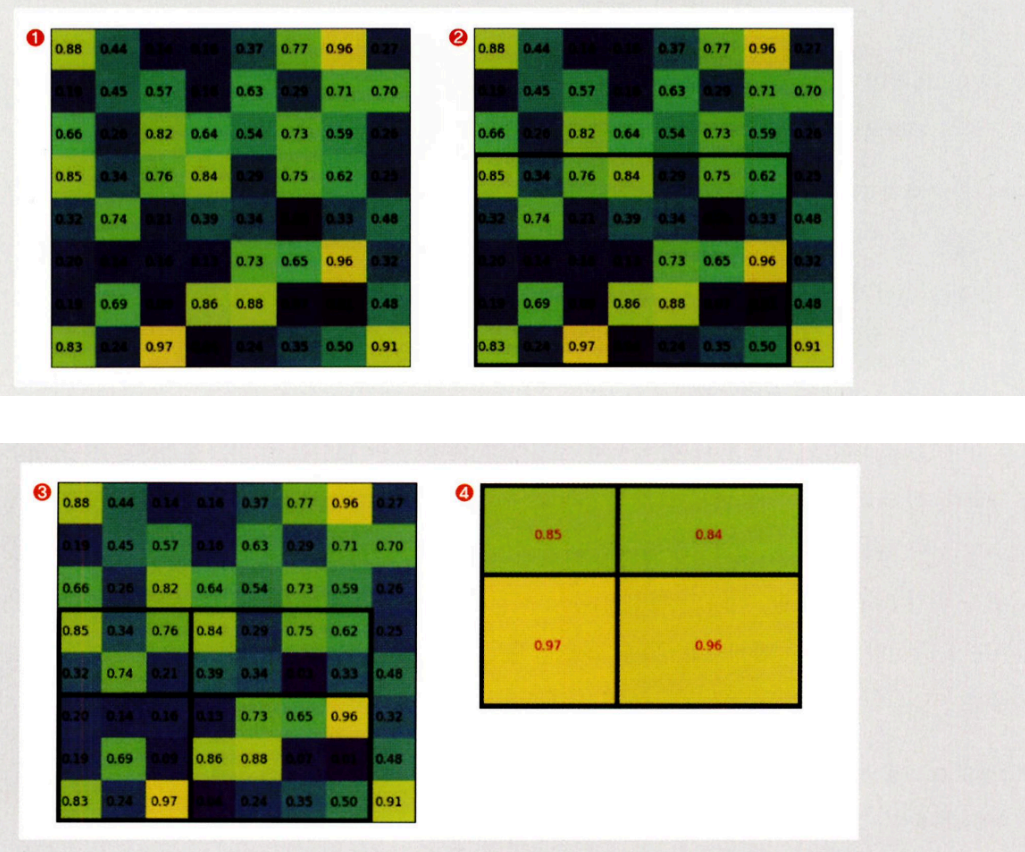
⇒ Fast R-CNN은 R-CNN의 속도 문제를 개선하려고 RoI 풀링을 도입

RoI 풀링

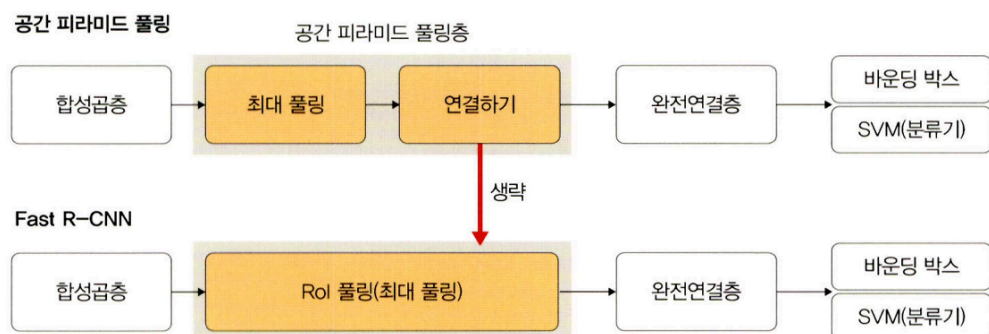
- 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결괏값 크기를 동일하게 맞추는 방법

예를 들어 다음 그림과 같이 박스 한 개가 픽셀 한 개를 뜻하는 특성 맵이 있다고 합시다. 즉, 8×8 특성 맵(❶)에서 선택적 탐색으로 뽑아냈던 7×5 후보 영역(❷)이 있으며, 이것을 2×2로 만들기 위해 스트라이드(7/2=3, 5/2=2)로 폴링 영역(❸)을 정하고 최대 폴링을 적용하면 2×2 결과(❹)를 얻을 수 있습니다.

▼ 그림 6-42 Roi 폴링



Fast R-CNN의 개념



- 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 함
- 최종 CNN 특성 맵은 폴링을 적용하여 완전연결층을 통과하도록 크기를 조정

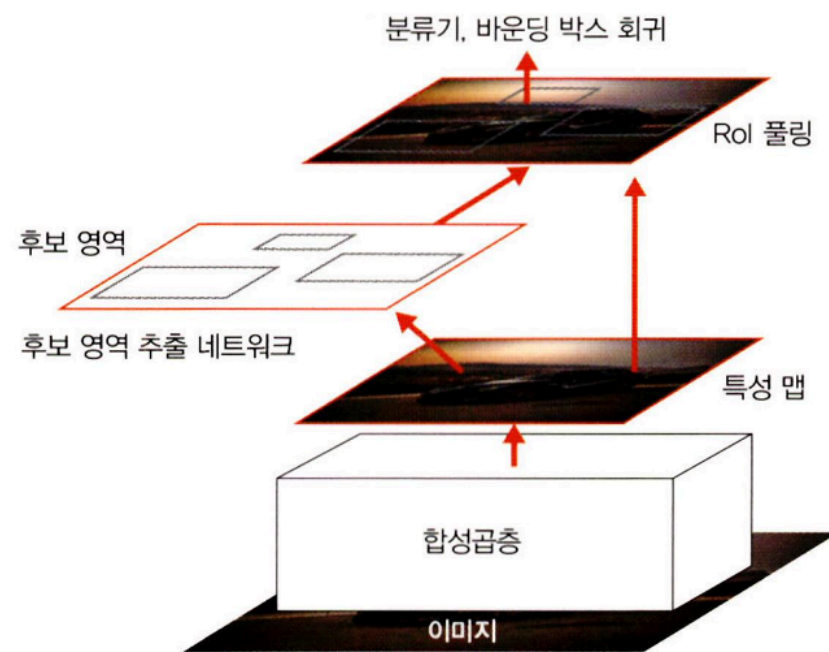
⇒ 바운딩 박스마다 CNN을 돌리는 시간을 단축

6.2.4. Faster R-CNN

Faster R-CNN 개념

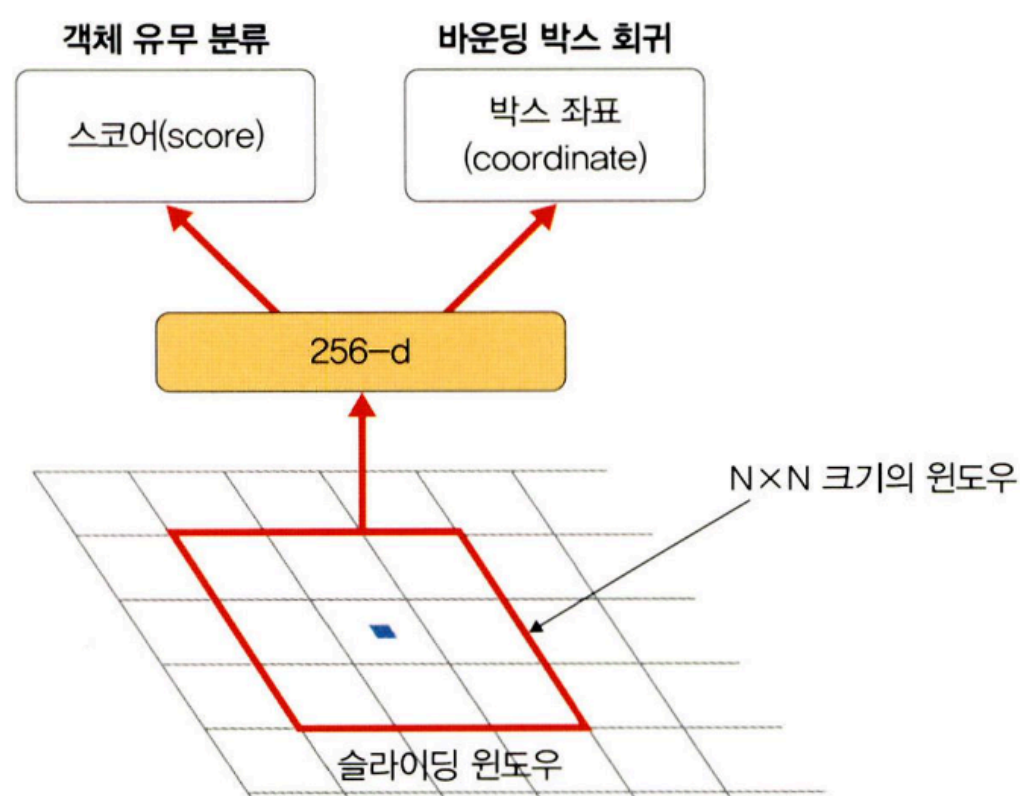
- 기존 Fast R-CNN 속도의 걸림돌이었던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계
 - ⇒ 기존 Fast R-CNN + 후보 영역 추출 네트워크
- 외부의 느린 선택적 탐색(CPU로 계산) 대신 내부의 빠른 RPN(GPU로 계산)을 사용

구조



- RPN: 마지막 합성곱층 다음에 위치
- 그 뒤에 Fast R-CNN과 마찬가지로 RoI 풀링, 분류기, 바운딩 박스 회귀가 위치
- 바운딩 박스 회귀: 바운딩 박스가 이미지의 객체를 정확히 포착하여 포함시킬 수 있도록 조정해주는 역할

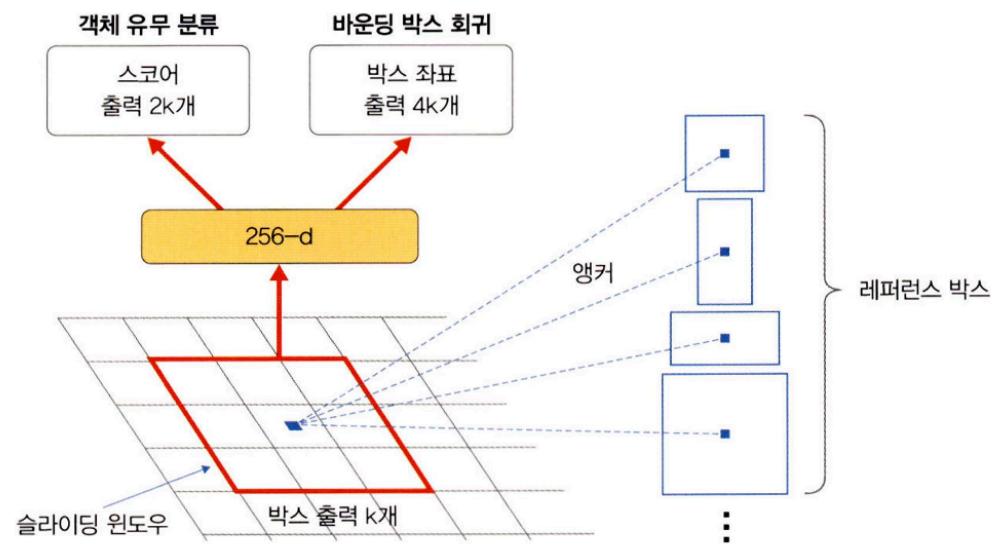
후보 영역 추출 네트워크



- 입력: 특성 맵 NxN 크기의 작은 윈도우 영역
→ 해당 영역에 객체의 존재 유무 판단을 위해 이진 분류를 수행하는 작은 네트워크 생성
- 바운딩 박스 회귀(R-CNN, Fast R-CNN에서 사용) 또한 위치 보정(좌표점 추론)을 위해 추가
- 하나의 특성 맵에서 모든 영역에 대한 객체의 존재 유무를 확인하기 위해 슬라이딩 윈도우 방식으로 앞서 설계한 작은 윈도우 영역(NxN 크기)을 이용하여 객체 탐색

⇒ 이미지에 존재하는 객체들의 크기와 비율이 다양하기 때문에 고정된 NxN 크기의 입력만으로 다양한 크기와 비율의 이미지를 수용하기 어려움

앵커



- 후보 영역 추출 네트워크의 단점을 보완하기 위해 사용
- 여러 크기와 비율의 레퍼런스 박스 k개를 미리 정의, 각각의 슬라이딩 윈도우 위치마다 박스 k개를 출력하도록 설계
- 후보 영역 추출 네트워크의 출력 값은 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 2k개의 분류에 대한 출력과 x, y, w, h 위치 보정 값을 위한 4k개의 회귀 출력을 가짐
 - 예) 특성 맵 크기가 $w \times h$ 라면, 하나의 특성 맵에 앵커가 총 $w \times h \times k$ 개 존재

6.3. 이미지 분할을 위한 신경망

6.3.0. 이미지 분할(image segmentation)

- 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것
- 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출

6.3.1. 완전 합성곱 네트워크(Fully Convolutional Network, FCN)

완전연결층의 한계

- 고정된 크기의 입력만 받아들임
- 완전연결층을 거친 후에는 위치 정보가 사라짐

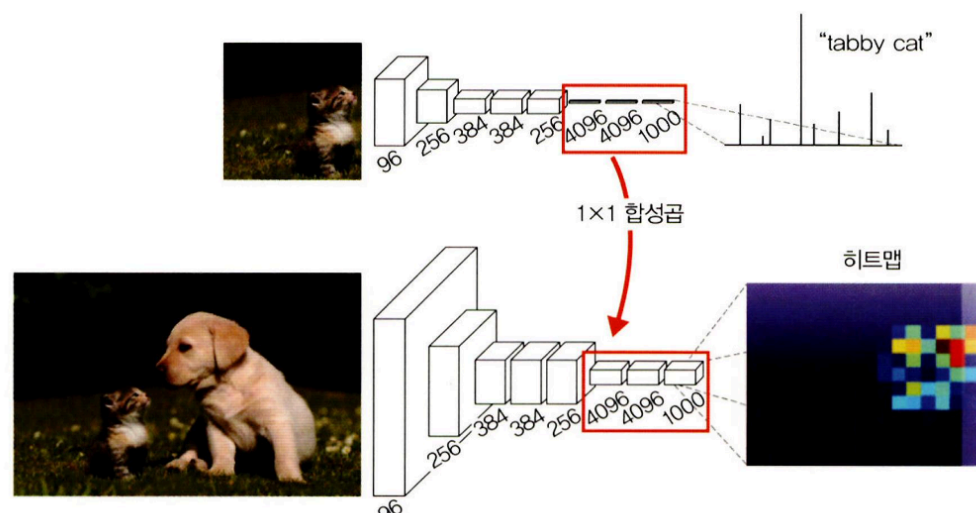
⇒ 이러한 문제를 해결하기 위해 완전 합성곱 네트워크 사용

완전 합성곱 네트워크 개념

- 완전연결층을 1×1 합성곱으로 대체
- 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크

예시

- AlexNet 아래쪽에서 사용되었던 완전연결층 세 개를 1×1 합성곱으로 변환
 - ⇒ 위치 정보가 남아 있기 때문에 히트맵 그림과 같이 고양이의 위치를 확인할 수 있음



장점

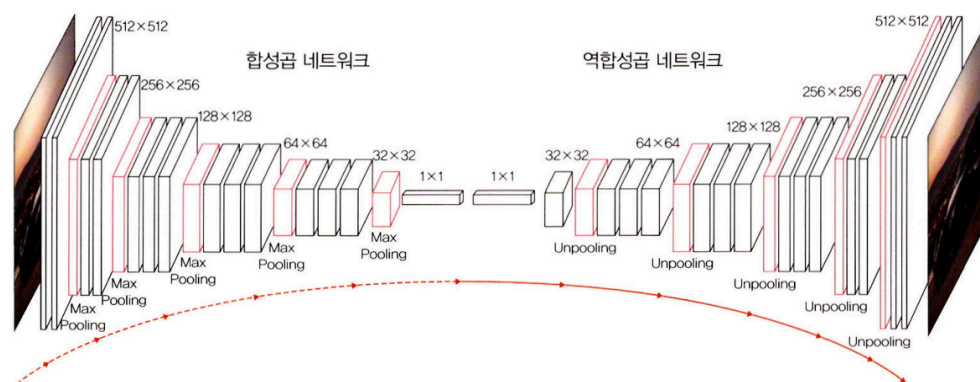
- 합성곱층으로 사용되기 때문에 입력 이미지에 대한 크기 제약이 사라짐

단점

- 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐
- 낮아진 해상도를 복원하기 위해 업 샘플링(최종 이미지 크기와 입력 이미지 크기를 같도록 하는 것) 방식을 사용
⇒ 이미지의 세부 정보를 잃어버림

6.3.2. 합성곱 & 역합성곱 네트워크(convolutional&deconvolutional network)

- 완전 합성곱 네트워크의 장점을 해결하기 위해 도입



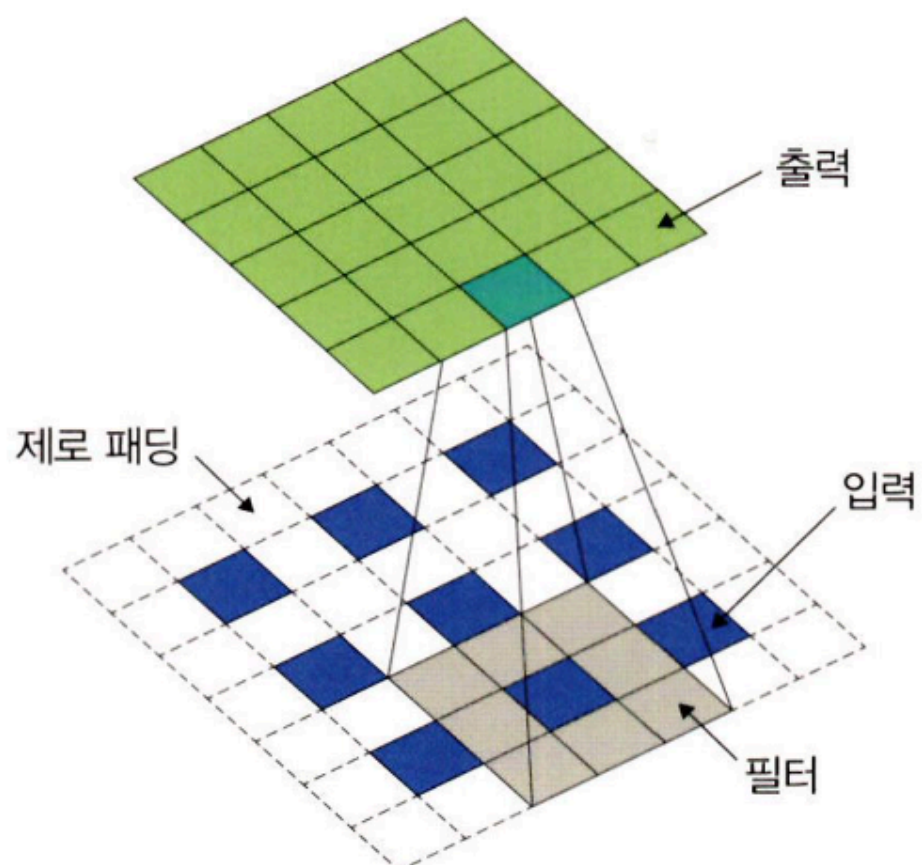
역합성곱[업 샘플링] 개념

- CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용
- 시맨틱 분할 등에 활용

역합성곱 동작

- CNN에서 합성곱층은 합성곱을 사용하여 특성 맵 크기를 줄임. 역합성곱은 반대로 특성 맵 크기를 증가시키는 방식으로 동작

- 각각의 픽셀 주위에 제로 패딩을 추가
- 패딩된 것에 합성곱 연산을 수행



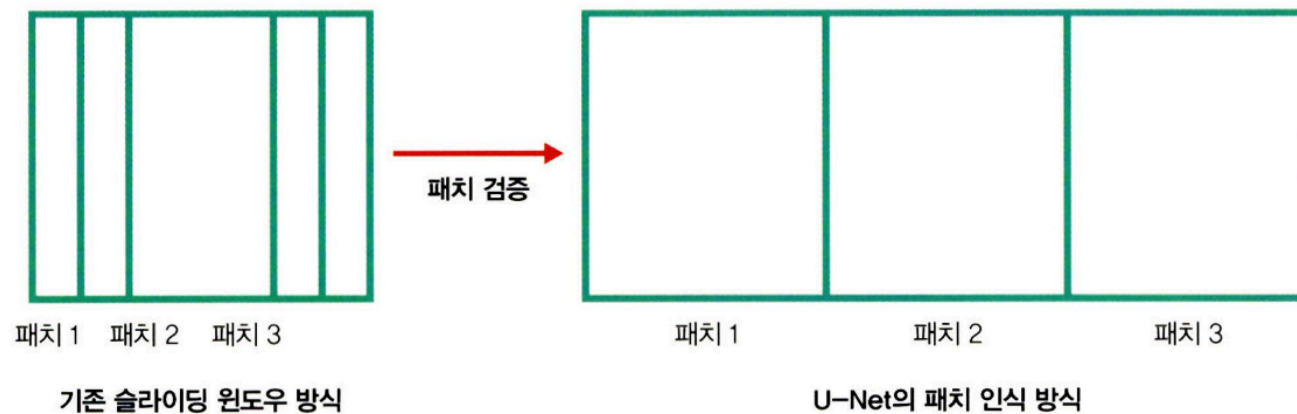
- 아래쪽의 파란 픽셀: 입력
- 초록 픽셀: 출력
- 파란 픽셀 주위로 흰색 제로 패딩 수행 → 회색 필터로 합성곱 연산을 수행 → 초록 픽셀 출력

6.3.3. U-Net

- 바이오 메디컬 이미지 분할을 위한 합성곱 신경망

특징

1. 속도가 빠름

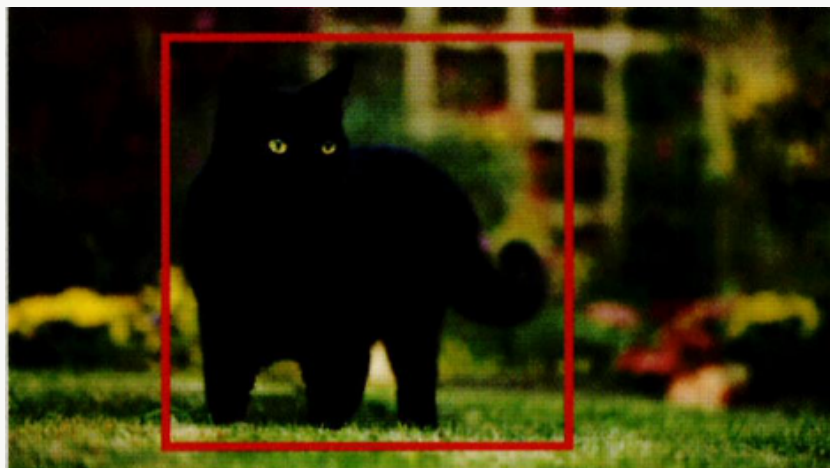


- 기존 슬라이딩 윈도우 방식을 이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증하기 때문에 속도가 느림
- U-Net은 이미 검증이 끝난 패치는 건너뛰기 때문에 속도가 빠름

2. 트레이드오프에 빠지지 않음

- 일반적으로 패치 크기가 커진다면 넓은 범위의 이미지를 인식하는 데 뛰어나기 때문에 컨텍스트 인식에 탁월. 하지만 지역화에 한계가 있음
⇒ 너무 넓은 범위를 한 번에 인식하기 때문에 지역화에는 약함
- U-Net은 컨텍스트 인식과 지역화 트레이드오프 문제를 개선

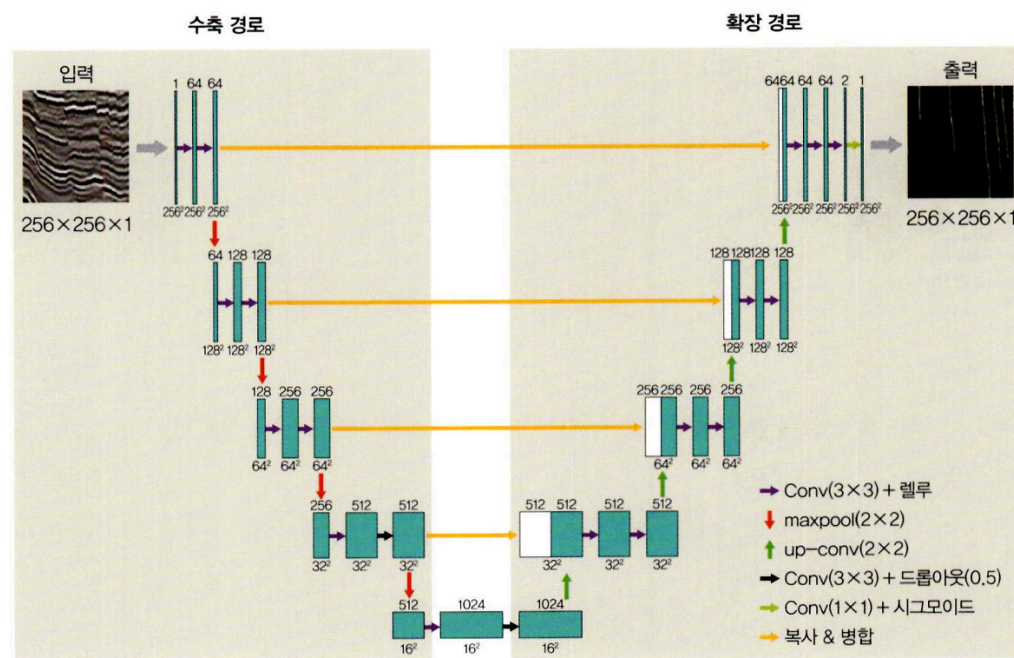
지역화



- 이미지 안에 객체 위치 정보를 출력해 주는 것
- 주로 바운딩 박스를 많이 사용
- 바운딩 박스의 네 꼭짓점 픽셀 좌표가 출력되는 것이 아닌 왼쪽 위, 오른쪽 아래 좌표를 출력

U-Net 구조

- FCN을 기반으로 구축
- 수축 경로(contracting path)와 확장 경로(expansive path)로 구성
 - 수축 경로: 컨텍스트를 포착
 - 확장 경로: 특성 맵을 업 샘플링, 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합하여 정확한 지역화를 수행
- 3×3 합성곱이 주를 이룸
- 각 합성곱 블록은 3x3 합성곱 두 개로 구성되어 있으며, 그 사이에 드롭아웃이 있음



- 수축 경로
 - 블록이 3x3 합성곱 두 개로 구성된 것이 네 개가 있는 형태
 - 각 블록은 최대 풀링을 이용하여 크기를 줄이면서 다음 블록으로 넘어감
- 확장 경로
 - 합성곱 블록에 up-conv를 앞에 붙임
 - 수축 과정에서 줄여든 크기를 다시 키워 가면서 합성곱 블록을 이용하는 형태
- 크기가 다양한 이미지의 객체를 분할하기 위해 크기가 다양한 특성 맵을 병합할 수 있도록 다운 샘플링과 업 샘플링을 순서대로 반복하는 구조

6.3.4. PSPNet

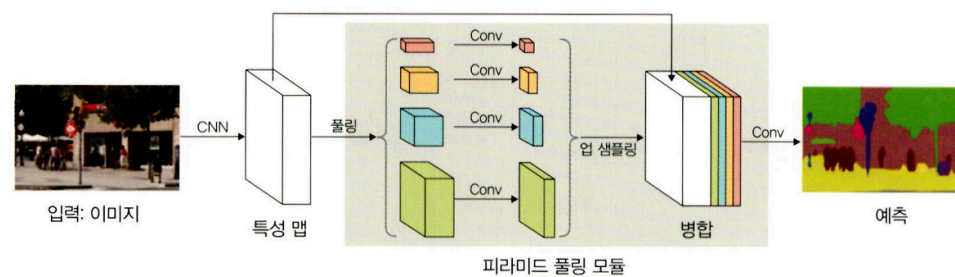
- 시맨틱 분할 알고리즘
- 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈을 추가

훈련 과정

1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링을 함
 - 1×1, 2×2, 3×3, 6×6 크기로 풀링을 수행
 - 1×1 특성 맵: 가장 광범위한 정보를담음
 - 각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담음
2. 1×1 합성곱을 사용하여 채널 수를 조정
 - 풀링층 개수를 N이라고 할 때, 출력 채널 수 = 입력 채널 수/N
3. 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링
 - 양선형 보간법 사용

- 보간법(Interpolation)
 - 화소 값을 할당받지 못한 영상의 품질은 안 좋을 수밖에 없는데, 이때 빈 화소에 값을 할당하여 좋은 품질의 영상을 만드는 방법
 - 선형 보간법과 양선형 보간법이 있음
- 선형 보간법(linear interpolation)
 - 원시 영상의 화소 값 두 개를 사용하여 원하는 좌표에서 새로운 화소 값을 계산하는 방법
- 양선형 보간법(bilinear interpolation)
 - 화소당 선형 보간을 세 번 수행
 - 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해서 얻음

4. 원래의 특성 맵과 1~3에서 생성한 새로운 특성 맵들을 병합

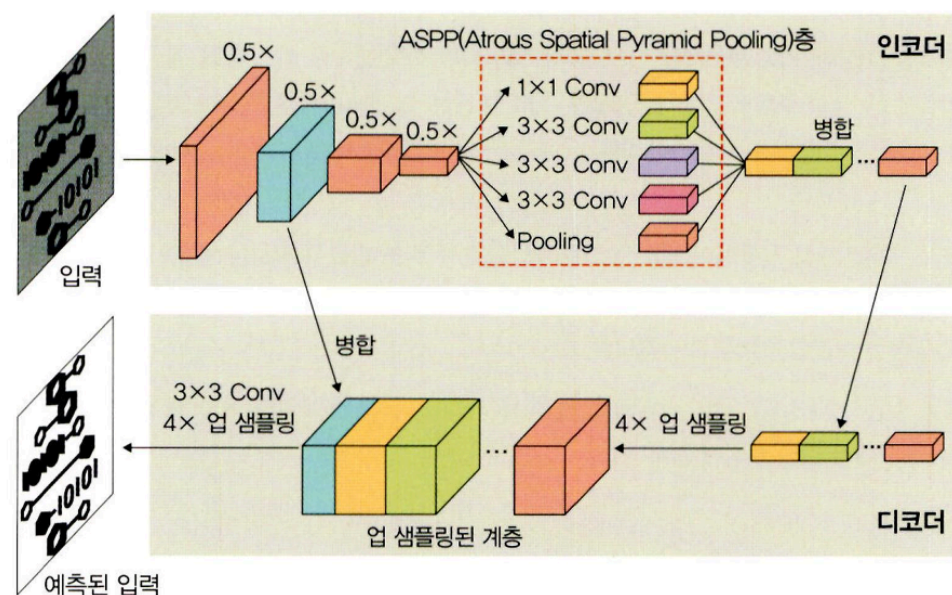


풀링 4개를 사용했지만, 구현에 따라 다르게 설정 가능

6.3.5. DeepLabv3/DeepLab3v+

- 완전연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크

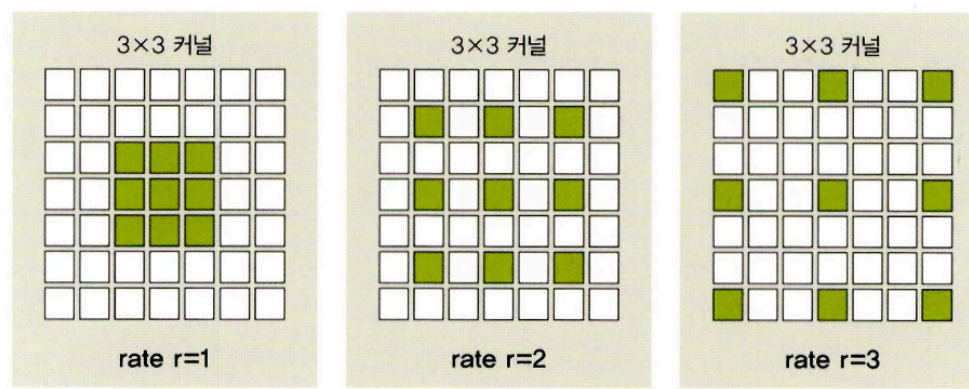
구조



- 인코더와 디코더 구조
- 일반적으로 인코더-디코더 구조에서는 불가능했던 인코더에서 추출된 특성 맵의 해상도를 Atrous 합성곱을 도입하여 제어할 수 있도록 했음

Atrous 합성곱

- 필터 내부에 빈 공간을 둔 채로 작동
- rate



- 얼마나 많은 빈 공간을 가질지 결정하는 파라미터
- rate $r = 1$ 일 경우 기존 합성곱과 동일하게 빈 공간을 가짐
- r 이 커질수록 빈 공간은 더 많아짐
- 이미지 분할에서 높은 성능을 내려면 수용 영역의 크기가 중요. 수용 영역을 확대하여 특성을 찾는 범위를 넓게 해 주기 때문.
⇒ Atrous 합성곱을 활용하면 파라미터 수를 늘리지 않으면서도 수용 영역을 크게 키울 수 있기 때문에 이미지 분할 분야에서 많이 사용

수용영역

- 외부 자극이 전체에 영향을 주는 것이 아니라 특정 영역에만 영향을 준다는 의미
- 영상에서 특정 위치에 있는 픽셀들은 그 주변에 있는 일부 픽셀과 연관성은 높지만 거리가 멀어질수록 그 영향은 감소
- 영상 전체 영역에 대해 서로 동일한 중요도를 부여하여 처리하는 대신, 특정 범위를 한정해서 처리하여 효과적으로 훈련을 수행하는 것

- 일반적인 CNN을 적용하면 출력은 입력에 비해 1/32로 줄어들지만, Atrous 합성곱을 적용하면 1/8로 줄어듬.
⇒ 특성 맵 크기가 기존 대비 4배 보존

